

GPU-Accelerated Monte Carlo Option Pricing Engine — Daniel Kim

GPU-Accelerated Monte Carlo Option Pricing Engine

EN605.617 GPU Programming — Final Project Report

Author: Daniel Kim

Date: May 2026

GPU: NVIDIA GeForce RTX 3060 Ti (Compute 8.6, 38 SMs, 4864 CUDA cores, 8 GB VRAM)

1. Introduction

Financial option pricing is a computationally intensive problem. A Monte Carlo simulation estimates an option's fair value by generating thousands to millions of random asset price paths, computing the payoff at expiry for each path, and averaging the discounted result. For a single option this is trivially parallelizable — every simulated path is completely independent — making it an ideal candidate for GPU acceleration.

This project implements a fully-featured Monte Carlo option pricing engine in CUDA. The engine supports five distinct option types (European, Asian, Barrier, Digital, and Merton jump-diffusion variants), computes option Greeks, prices multi-option portfolios with concurrent CUDA streams, renders GPU-generated visualizations (spaghetti plots and payoff histograms), and provides an interactive ncurses terminal dashboard. The project served as a vehicle for applying and benchmarking the full range of GPU programming techniques covered in the course.

2. Background and Motivation

2.1 The Black-Scholes Model

Under the Black-Scholes model, asset prices follow Geometric Brownian Motion (GBM):

$$dS = r \cdot S \cdot dt + \sigma \cdot S \cdot dW$$

where r is the risk-free rate, σ is volatility, and dW is a Wiener process increment. The discrete-time Euler-Maruyama approximation used in simulation is:

$$S_{t+\Delta t} = S_t \cdot \exp\left((r - \sigma^2/2) \cdot \Delta t + \sigma \cdot \sqrt{\Delta t} \cdot Z\right)$$

where $Z \sim N(0,1)$.

For European calls, the Black-Scholes analytical price is:

$$\begin{aligned} C &= S \cdot N(d_1) - K \cdot e^{-rT} \cdot N(d_2) \\ d_1 &= [\ln(S/K) + (r + \sigma^2/2) \cdot T] / (\sigma \cdot \sqrt{T}) \\ d_2 &= d_1 - \sigma \cdot \sqrt{T} \end{aligned}$$

Monte Carlo estimation approximates this by averaging $\max(S_T - K, 0)$ over N simulated paths and discounting: $C \approx e^{-rT} \cdot (1/N) \sum \max(S_T^{(i)} - K, 0)$.

2.2 Merton Jump-Diffusion

The Merton (1976) jump-diffusion model extends GBM with a compound Poisson jump process to

capture crashes and black swan events:

$$dS/S = (r - \lambda k) \cdot dt + \sigma \cdot dW + J \cdot dN$$

where λ is jump intensity (expected jumps/year), $k = E[e^J - 1]$, and $J \sim N(\mu_J, \sigma_J^2)$ is the log-normal jump size. The compensated drift $(r - \lambda k)$ ensures risk-neutral pricing. The per-timestep implementation checks $U \sim \text{Uniform}(0,1) < \lambda \cdot \Delta t$ to determine if a jump occurs.

3. System Architecture

The engine is organized into nine source modules:

Module	File	Responsibility
Entry point	main.cu	CLI parsing, output formatting, orchestration
MC simulation	monte_carlo.cu	Unified kernel: GBM + jump-diffusion, all option types
Reduction	reduction.cu	Shared-memory parallel sum reduction + variance
Black-Scholes	black_scholes.cu	Analytical pricing and Greeks for validation
Greeks	greeks.cu	GPU bump-and-reprice finite difference Greeks
Streams	streams.cu	Multi-option portfolio pricing via CUDA streams
Visualization	render_ppm.cu	GPU spaghetti plot and payoff histogram rendering
Dashboard	dashboard.cu	ncurses interactive terminal dashboard
GL Dashboard	gl_dashboard.cu	OpenGL real-time graphical dashboard

Data flow: 1. CLI args $\rightarrow$ OptionParams struct 2. OptionParams $\rightarrow$ constant memory (cudaMemcpyToSymbol) 3. cuRAND states initialized per-thread (curand_init) 4. MC kernel: each thread simulates one path, writes payoff to global memory 5. Shared-memory reduction: mean and variance of payoffs 6. Discounting $\rightarrow$ final option price and confidence interval

4. GPU Programming Techniques

4.1 Thread/Block/Grid Configuration

Each simulated price path maps to exactly one CUDA thread:

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
if (idx >= num_paths) return;
```

Grid size is computed as $\text{ceil}(\text{num_paths} / \text{block_size})$. The default block size of 256 was selected empirically via a block-size sweep benchmark (see Section 6.2). At 256 threads/block, the RTX 3060 Ti achieves 604 Mpaths/sec — the highest throughput observed.

4.2 Constant Memory

Option parameters (spot, strike, rate, volatility, expiry, barrier, option type, jump parameters) are stored in CUDA constant memory using a `__constant__ OptionParams d_params` symbol. These 13 values are broadcast to all threads every timestep, exploiting the constant memory cache:

```
__constant__ OptionParams d_params;

void mc_set_params(const OptionParams *params) {
    cudaMemcpyToSymbol(d_params, params, sizeof(OptionParams));
}
```

4.3 Register Pressure and Per-Thread State

At the start of the kernel, all frequently-accessed parameters are loaded from constant memory into

registers:

```
float S = d_params.S;
float K = d_params.K;
float sigma = d_params.sigma;
...
curandState local_state = states[idx];
```

The cuRAND state is also kept in a local (register-spilled) variable for the duration of the path simulation, then written back once at the end — minimizing global memory traffic.

4.4 Global Memory

Per-thread payoff results are written to a device-side `float *d_payoffs` array in global memory. This is the only global memory write per thread per kernel invocation, keeping memory bandwidth pressure low.

4.5 Shared Memory Reduction

The custom reduction in `reduction.cu` uses shared memory for per-block partial sums, avoiding global atomic contention. Two elements are loaded per thread on the first step, halving the required blocks:

```
__shared__ float sdata[256];
float val = 0.0f;
if (idx < n) val += input[idx];
if (idx + blockDim.x < n) val += input[idx + blockDim.x];
sdata[tid] = val;
__syncthreads();

for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
    if (tid < s) sdata[tid] += sdata[tid + s];
    __syncthreads();
}
```

The reduction is applied iteratively until a single scalar sum remains. A second pass computes variance by first computing $(x - \text{mean})^2$ for each element, then re-reducing.

4.6 cuRAND

Each thread maintains its own independent `curandState`, initialized with a unique sequence number (`curand_init(seed, idx, 0, &states[idx])`). During path simulation, each timestep draws $Z = \text{curand_normal}(\&\text{local_state})$ for the GBM diffusion term. Jump-diffusion paths additionally draw $U = \text{curand_uniform}(\&\text{local_state})$ to test for a jump event, and $Z_J = \text{curand_normal}(\&\text{local_state})$ for the jump magnitude.

4.7 CUDA Streams

The portfolio pricing mode (`--portfolio`) prices 8 options concurrently using 4 CUDA streams. Each stream independently: 1. Initializes cuRAND states (`init_curand_stream`) 2. Runs the MC kernel (`mc_stream_kernel`) with option-specific parameters passed via registers (not constant memory, since streams require different params simultaneously) 3. Asynchronously copies payoffs back to pinned host memory (`cudaMemcpyAsync`)

The `mc_stream_kernel` receives `OptionParams` by value, loading it into thread registers, rather than using constant memory (which is shared globally). Pinned host memory (`cudaMallocHost`) is used for the async copies to enable true DMA overlap.

4.8 CUDA Event Timing

CUDA events bracket both the full execution (RNG init + kernel + reduction) and the kernel alone:

```
cudaEventRecord(start_total);
curandState *d_states = mc_init_rng(num_paths, block_size);
```

```

cudaEventRecord(start_kernel);
float *d_payoffs = mc_run_simulation(d_states, num_paths, num_steps, block_size);
cudaEventRecord(stop_kernel);
cudaEventSynchronize(stop_kernel);
reduce_payoffs(d_payoffs, num_paths, &mean_payoff, &variance);
cudaEventRecord(stop_total);

```

This separates kernel time from RNG initialization overhead, which is significant at low path counts.

5. Supported Option Types

The unified kernel handles all option types via a switch on `d_params.type`:

Option Type	Payoff Formula
European Call	$\max(S_T - K, 0)$
European Put	$\max(K - S_T, 0)$
Asian Call	$\max(\text{avg}(S) - K, 0)$
Asian Put	$\max(K - \text{avg}(S), 0)$
Barrier Up-and-Out Call	$\max(S_T - K, 0)$ if never touched barrier H above
Barrier Down-and-Out Call	$\max(S_T - K, 0)$ if never touched barrier L below
Digital Call	1 if $S_T > K$, else 0
Digital Put	1 if $S_T < K$, else 0

Asian options accumulate the running sum `sum_S` during path simulation; barrier options maintain a `barrier_hit` flag. Both incur minimal overhead since all option types execute the same GBM loop.

6. Performance Results

6.1 CPU vs. GPU Speedup

The CPU baseline (`cpu_baseline.cpp`) runs single-threaded GBM simulation using `rand()` and the standard Box-Muller transform. Measured on the same machine:

Paths	CPU (ms)	GPU Total (ms)	Speedup
10,000	83.07	0.38	217×
100,000	831.84	1.55	536×
1,000,000	8,312.23	18.02	461×

GPU total time includes RNG initialization and reduction. Peak speedup of 536× is observed at 100K paths, where GPU kernel time is minimal and the CPU pays a full millisecond per 10K paths. At 1M paths, GPU total time rises due to RNG init overhead while remaining orders of magnitude faster than the CPU.

6.2 Block Size Sweep (1M paths, European Call)

Block Size	Kernel (ms)	Throughput (Mpaths/s)
32	2.33	429
64	2.24	446
128	1.72	582
256	1.65	605
512	1.67	600
1024	1.72	580

Block size 256 achieves peak throughput of 605 Mpaths/sec. This aligns with the hypothesis in the proposal: uniform control flow (every thread follows the same loop body) makes MC paths ideal for

the 256-thread sweet spot, which maximizes register utilization and SM occupancy on the RTX 3060 Ti without over-subscribing shared resources.

6.3 Monte Carlo Convergence

Convergence toward the Black-Scholes analytical price follows the expected $O(1/\sqrt{N})$ rate:

Paths	MC Price	BS Price	Error (%)	Std Error
1,000	11.2850	10.4506	7.96	0.4394
10,000	10.5448	10.4506	0.90	0.1458
100,000	10.4715	10.4506	0.20	0.0469
1,000,000	10.4642	10.4506	0.13	0.0147
5,000,000	10.4517	10.4506	0.01	0.0066
10,000,000	10.4507	10.4506	0.01	0.0047

At 1M paths, the GPU achieves 0.13% error vs. the Black-Scholes formula in 1.56 ms (kernel only). At 10M paths, the observed error was below 0.01% for this run, with standard error continuing to scale approximately as $O(1/\sqrt{N})$ as expected from Monte Carlo theory.

7. Greeks via Bump-and-Reprice

Option Greeks (sensitivities) are computed via central finite differences, each requiring additional GPU MC runs:

Greek	Definition	Bump Size
Delta	$\partial V / \partial S$	$dS = 1\% \times S$ (central)
Gamma	$\partial^2 V / \partial S^2$	$dS = 1\% \times S$ (second-order central)
Vega	$\partial V / \partial \sigma$	$d\sigma = 0.01$ (central)
Theta	$\partial V / \partial t$	$dT = 1/365$ (forward)
Rho	$\partial V / \partial r$	$dr = 0.001$ (central)

Delta and Gamma are computed from the same two perturbed prices (`price_up`, `price_down`). Each Greek requires 2 extra MC runs (except Gamma which reuses Delta's), so computing all 5 Greeks costs 8 full MC kernel invocations total. This is still sub-second for 1M paths per run.

MC Greeks for European calls closely match the Black-Scholes analytical Greeks, confirming correctness.

8. Visualizations

Two GPU-assisted visualizations are generated by `render_ppm.cu`:

8.1 Spaghetti Plot (Price Path Visualization)

Each thread simulates one GBM price path and draws it pixel-by-pixel into a shared 1920×1080 framebuffer using direct writes (last-writer-wins — races produce a natural blending effect that is visually acceptable for this use case). Paths above the strike are drawn in green (ITM); paths below in red (OTM). The result immediately shows the distribution of outcomes and the GPU's massive parallelism — 5,000 paths rendered simultaneously. Note that the spaghetti kernel simulates pure GBM; it does not incorporate jump-diffusion parameters.

8.2 Payoff Histogram

A `histogram_kernel` exists in `render_ppm.cu` that partitions payoffs into bins using `atomicAdd` per thread. The rendered PPM histogram used in `render_histogram_ppm()` performs the binning on the host after

copying payoffs back from device memory, then draws the bars on the CPU. For European calls under Black-Scholes, the histogram reveals the characteristic mix of zero payoffs (OTM paths) and an approximately log-normal tail (ITM paths).

Both images are output as PPM files (convertible to PNG with ImageMagick).

9. Interactive Dashboards

Two interactive modes make the GPU parallelism tangible in real time.

9.1 OpenGL Graphical Dashboard (--gui)

The OpenGL dashboard (`gl_dashboard.cu`) runs a continuous pricing loop and renders the results live using GLFW + GLEW:

- **Price history chart** — plots MC price across iterations, color-coded green (above running average) or red (below), with a dashed yellow Black-Scholes reference line
- **Payoff distribution histogram** — live bar chart of the payoff distribution across all paths, updating every iteration
- **Parameters panel** — spot, strike, volatility, rate, expiry, jump parameters
- **Results panel** — MC price, running average, 95% confidence interval, BS analytical price, error %, kernel time, throughput, iteration count, and all 5 Greeks (computed every 10 iterations via bump-and-reprice)

Every dashboard frame represents a full GPU MC simulation. With 500,000 paths and a 1.13ms kernel time, the dashboard runs at **~100 iterations/second**, meaning in one second the GPU has simulated **50 million independent price paths in parallel**. The GPU maps paths to CUDA threads and schedules those threads concurrently across 38 SMs, reaching a measured throughput of 442 Mpaths/sec on the RTX 3060 Ti.

To put it concretely: in the time it takes to blink (~150ms), the GPU completes ~65 million simulated price paths. The CPU doing the same work sequentially would take over 9 minutes.

Keyboard controls allow live parameter changes: +/- adjusts volatility, </> moves spot price, C/P switches call/put, A prices Asian, D prices Digital, J toggles Merton jump-diffusion, and 1-9 scales path count from 10K to 10M.

9.2 ncurses Terminal Dashboard (--dashboard)

The ncurses dashboard provides the same live repricing loop in a terminal UI, with ASCII sparklines for price and timing history, Greek readouts, and keyboard parameter adjustment — useful on headless servers without a display.

10. Presets and Jump-Diffusion Models

Six market presets are built in with realistic parameters derived from current market data:

Preset	Spot	Strike	Volatility
BTC	\$96,800	\$100,000	65%
ETH	\$1,820	\$2,000	75%
SPY	\$563	\$565	16%
AAPL	\$198	\$200	25%
TSLA	\$272	\$280	55%
NVDA	\$110	\$115	45%

Jump-diffusion profiles for crypto assets and market stress scenarios:

Profile λ (jumps/yr) μ_J (mean) σ_J (vol)

BTC	3.0	-10%	20%
Crypto	4.0	-12%	25%
Equity	0.5	-8%	10%
Crisis	1.0	-30%	15%
Mild	2.0	-3%	5%

The BTC + crisis jump-diffusion combination dramatically widens the payoff distribution compared to the standard GBM model, visible in the histogram output. The spaghetti plot visualization uses pure GBM paths and does not incorporate jump parameters.

11. Hypothesis Validation

Hypothesis	Result
100–500× GPU speedup	✓ Peak 536× at 100K paths
Block size 256 optimal	✓ 605 Mpaths/sec, highest observed
$O(1/\sqrt{N})$ convergence	✓ Std error scales as expected
Shared memory reduction effective	✓ Custom reduction avoids global atomics entirely
Stream overlap for portfolio	✓ 8 options priced concurrently on 4 streams
Visualization on GPU	✓ GPU path rendering demonstrated; final annotation and PPM output are CPU-side

12. Build and Usage

```
make all
./mc_pricer --help

# European call with default params (S=100, K=100,  $\sigma$ =20%, T=1yr)
./mc_pricer

# BTC with jump-diffusion
./mc_pricer --preset BTC --jumps btc

# Asian option
./mc_pricer --type asian

# Greeks
./mc_pricer --greeks

# Portfolio (8 options, 4 streams)
./mc_pricer --portfolio

# Interactive dashboard
./mc_pricer --dashboard

# Spaghetti + histogram visualization
./mc_pricer --visualize
```

Requirements: NVIDIA GPU (compute 8.0+), CUDA Toolkit 11.5+, GCC 11+, ncurses, OpenGL, GLFW3, and GLEW.

13. Source Code Organization

```
src/
├─ main.cu           CLI parsing, output formatting, main workflow
```

— monte_carlo.cu	Unified MC kernel (GBM + jump-diffusion, all types)
— reduction.cu	Shared-memory parallel reduction (mean + variance)
— black_scholes.cu	Analytical Black-Scholes pricing and Greeks
— greeks.cu	GPU bump-and-reprice finite-difference Greeks
— streams.cu	Portfolio pricing via concurrent CUDA streams
— render_ppm.cu	GPU spaghetti plot + payoff histogram rendering
— cpu_baseline.cpp	Single-threaded CPU reference implementation
— dashboard.cu	ncurses interactive terminal dashboard
— gl_dashboard.cu	OpenGL real-time graphical dashboard
include/	
— option_params.h	OptionParams struct and OptionType enum

14. Conclusion

This project demonstrates that Monte Carlo option pricing is a natural fit for GPU acceleration. The single-threaded CPU implementation requires over 8 seconds to price 1M paths; the GPU completes the same task in 18 ms — a $461\times$ speedup. At 100K paths, the speedup peaks at $536\times$.

Beyond raw performance, the project applies the full range of EN605.617 techniques: constant memory for broadcast parameters, shared-memory reductions to avoid global atomic contention, cuRAND for GPU-native random number generation, CUDA streams for concurrent portfolio pricing, and CUDA event timing for rigorous benchmarking. The extension to exotic options (Asian, Barrier, Digital) and Merton jump-diffusion demonstrates that the unified kernel design scales to real-world derivative pricing complexity with minimal code overhead.

The GPU's parallelism is made visually tangible through the spaghetti plot: 5,000 independently simulated price paths rendered simultaneously in a 1920×1080 image — a workload that would require explicit serialization on the CPU.

References

1. Black, F. and Scholes, M. (1973). "The Pricing of Options and Corporate Liabilities." *Journal of Political Economy*, 81(3), 637-654.
2. Merton, R.C. (1976). "Option Pricing When Underlying Stock Returns Are Discontinuous." *Journal of Financial Economics*, 3(1-2), 125-144.
3. Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering*. Springer.
4. NVIDIA Corporation. *CUDA C++ Programming Guide*, Version 11.5.
5. NVIDIA Corporation. *cuRAND Library User Guide*, Version 11.5.